Abstract Interpretation & its Application to Semantics Approximation

An Introduction

Marco Ciccalè

marco.ciccale@imdea.org

Universidad Politécnica de Madrid (UPM), Madrid Spain
IMDEA Software Institute, Madrid, Spain

Aut. Coffee – March 12, 2026

Programs

Syntax

- We will work with a simple imperative programming language.
- We start by defining how programs must be *written*.

Syntax

- We will work with a simple imperative programming language.
- We start by defining how programs must be *written*.

Syntax of expressions

$$
\begin{array}{rcll}
\mathtt{x}, \mathtt{y}, \ldots & \in & \mathbb{V} & & \text{variables} \\
\mathtt{E} & \in & \mathbb{E} & ::= \quad \mathtt{1} \mid \mathtt{x} \mid \mathtt{E}_1 \ \mathtt{-} \ \mathtt{E}_2 & \text{arithmentic expressions} \\
 & & & \quad\ \mid \quad \mathtt{tt} \mid \mathtt{ff} \mid \mathtt{E}_1 \ \mathtt{<} \ \mathtt{E}_2 \mid \mathtt{E}_1 \ \mathtt{nand} \ \mathtt{E}_2 & \text{Boolean expressions}
\end{array}
$$

Syntax

- We will work with a simple imperative programming language.
- We start by defining how programs must be *written*.

Syntax of expressions

$$
\begin{array}{rcll}
\mathtt{x}, \mathtt{y}, \ldots & \in & \mathbb{V} & & \text{variables} \\
\mathtt{E} & \in & \mathbb{E} & ::= \ \mathtt{1} \mid \mathtt{x} \mid \mathtt{E}_1 \ \mathtt{-} \ \mathtt{E}_2 & \text{arithmentic expressions} \\
 & & & \mid \ \mathtt{tt} \mid \mathtt{ff} \mid \mathtt{E}_1 \ \mathtt{<} \ \mathtt{E}_2 \mid \mathtt{E}_1 \ \mathtt{nand} \ \mathtt{E}_2 & \text{Boolean expressions}
\end{array}
$$

Syntax of statements

$$
\mathtt{S} \ \in \ \mathbb{S} \ ::= \ \mathtt{skip} \mid \mathtt{x} \ \mathtt{:=} \ \mathtt{E} \mid \mathtt{S}_1 \mathtt{;} \ \mathtt{S}_2 \mid \mathbf{if} \ \mathtt{(E)} \ \mathbf{then} \ \mathtt{S}_1 \ \mathbf{else} \ \mathtt{S}_2 \mid \mathbf{while} \ \mathtt{(E)} \ \mathbf{do} \ \mathtt{S}
$$

Syntax

- We will work with a simple imperative programming language.
- We start by defining how programs must be *written*.

Syntax of expressions

$$
\begin{array}{rcll}
\mathtt{x}, \mathtt{y}, \ldots & \in & \mathbb{V} & \text{variables} \\
\mathtt{E} & \in & \mathbb{E} \quad ::= \quad \mathtt{1} \mid \mathtt{x} \mid \mathtt{E}_1 \ \mathtt{-} \ \mathtt{E}_2 & \text{arithmentic expressions} \\
 & & \quad\quad\quad\quad \mid \quad \mathtt{tt} \mid \mathtt{ff} \mid \mathtt{E}_1 \ \mathtt{<} \ \mathtt{E}_2 \mid \mathtt{E}_1 \ \mathtt{nand} \ \mathtt{E}_2 & \text{Boolean expressions}
\end{array}
$$

Syntax of statements

$$
\mathtt{S} \quad \in \quad \mathbb{S} \quad ::= \quad \mathtt{skip} \mid \mathtt{x} \ \mathtt{:=} \ \mathtt{E} \mid \mathtt{S}_1 \mathtt{;} \ \mathtt{S}_2 \mid \mathbf{\mathtt{if}} \ \mathtt{(E)} \ \mathbf{\mathtt{then}} \ \mathtt{S}_1 \ \mathbf{\mathtt{else}} \ \mathtt{S}_2 \mid \mathbf{\mathtt{while}} \ \mathtt{(E)} \ \mathbf{\mathtt{do}} \ \mathtt{S}
$$

- Purely *syntactic* object, it has no *meaning* (*i.e.*, no semantics) yet.

Syntax

- We will work with a simple imperative programming language.
- We start by defining how programs must be *written*.

Syntax of expressions

$$
\begin{array}{rcll}
\mathtt{x}, \mathtt{y}, \ldots & \in & \mathbb{V} & & \text{variables} \\
\mathtt{E} & \in & \mathbb{E} & ::= \ \mathtt{1} \mid \mathtt{x} \mid \mathtt{E}_1 \ \mathtt{-} \ \mathtt{E}_2 & \text{arithmetic expressions} \\
 & & & \mid \ \mathtt{tt} \mid \mathtt{ff} \mid \mathtt{E}_1 \ \mathtt{<} \ \mathtt{E}_2 \mid \mathtt{E}_1 \ \mathtt{nand} \ \mathtt{E}_2 & \text{Boolean expressions}
\end{array}
$$

Syntax of statements

$$
\mathtt{S} \ \in \ \mathbb{S} \ ::= \ \mathtt{skip} \mid \mathtt{x} \ \mathtt{:=} \ \mathtt{E} \mid \mathtt{S}_1 \mathtt{;} \ \mathtt{S}_2 \mid \mathbf{\mathtt{if}} \ \mathtt{(E)} \ \mathbf{\mathtt{then}} \ \mathtt{S}_1 \ \mathbf{\mathtt{else}} \ \mathtt{S}_2 \mid \mathbf{\mathtt{while}} \ \mathtt{(E)} \ \mathbf{\mathtt{do}} \ \mathtt{S}
$$

- Purely *syntactic* object, it has no *meaning* (*i.e.*, no semantics) yet.
- Defined *compositionally*, *i.e.*, composite elements such as $\mathtt{E}_1 \ \mathtt{-} \ \mathtt{E}_2$ have a unique decomposition into *strictly smaller* elements.

Semantics

- We now turn our heads to the meaning of programs: *semantics*.
- Semantics can come in many different flavors, and they complement each other.

Semantics

- We now turn our heads to the meaning of programs: *semantics*.
- Semantics can come in many different flavors, and they complement each other.

Operational [Plotkin, 1981]	**Denotational [Scott and Strachey, 1971]**	**Axiomatic [Hoare, 1969]**
Defines *how* a program is executed.	Models the *result* of executing a program.	Allows to prove (partial) correctess of a program.
Small- or *big-step* definitions.	*Compositional* definitions.	Assertion-based definitions.

Semantics

- We now turn our heads to the meaning of programs: *semantics*.
- Semantics can come in many different flavors, and they complement each other.

Operational [Plotkin, 1981]	**Denotational** [Scott and Strachey, 1971]	**Axiomatic** [Hoare, 1969]
Defines *how* a program is executed.	Models the *result* of executing a program.	Allows to prove (partial) correctess of a program.
Small- or *big-step* definitions.	*Compositional* definitions.	Assertion-based definitions.

- Herein, we concentrate in *denotational* semantics, but the approaches can be adapted to any semantics flavor.

Denotational semantics: expressions

- We consider the *structure* $(\mathbb{Z}, \mathbb{B}; 1, \mathtt{tt}, \mathtt{ff}, -, <, \uparrow)$ with the usual interpretations.
- And, for brevity, let $\mathbb{I} \triangleq \mathbb{Z} \cup \mathbb{B}$, and $\mathbb{E}\mathbb{v} \triangleq \mathbb{V} \to \mathbb{I}$.

[1] We use a dependent type to express that we require the expression $\mathtt{E}$ to have all of its variables considered in the (implicit) set of variables $\mathbb{V}$. A more detailed discussion on this matter can be found in [Cousot, 2021].

Denotational semantics: expressions

- We consider the *structure* $(\mathbb{Z}, \mathbb{B}; 1, \mathtt{tt}, \mathtt{ff}, -, <, \uparrow)$ with the usual interpretations.
- And, for brevity, let $\mathbb{I} \triangleq \mathbb{Z} \cup \mathbb{B}$, and $\mathbb{E}_{\mathbb{V}} \triangleq \mathbb{V} \to \mathbb{I}$.

Denotational semantics of expressions

$\mathscr{E} : \mathbb{V} \to (\mathtt{E} \in \mathbb{E}) \to \mathbb{E}_{\mathbb{V}} \rightharpoonup \mathbb{I}$ *where* $\mathtt{vars}(\mathtt{E}) \subseteq \mathbb{V}$[1]

$$
\begin{array}{llll}
\mathscr{E}[\![\mathtt{k}]\!]\sigma & \triangleq & k & \in \mathbb{I} \\
\mathscr{E}[\![\mathtt{x}]\!]\sigma & \triangleq & \sigma(\mathtt{x}) & \in \mathbb{I} \\
\mathscr{E}[\![\mathtt{E}_1 \ \mathtt{-} \ \mathtt{E}_2]\!]\sigma & \triangleq & v_1 - v_2 & \in \mathbb{Z} \quad \text{if } v_1 = \mathscr{E}[\![\mathtt{E}_1]\!]\sigma \in \mathbb{Z},\ v_2 = \mathscr{E}[\![\mathtt{E}_2]\!]\sigma \in \mathbb{Z} \\
\mathscr{E}[\![\mathtt{E}_1 \ \mathtt{<} \ \mathtt{E}_2]\!]\sigma & \triangleq & v_1 < v_2 & \in \mathbb{B} \quad \text{if } v_1 = \mathscr{E}[\![\mathtt{E}_1]\!]\sigma \in \mathbb{Z},\ v_2 = \mathscr{E}[\![\mathtt{E}_2]\!]\sigma \in \mathbb{Z} \\
\mathscr{E}[\![\mathtt{E}_1 \ \mathtt{nand} \ \mathtt{E}_2]\!]\sigma & \triangleq & v_1 \uparrow v_2 & \in \mathbb{B} \quad \text{if } v_1 = \mathscr{E}[\![\mathtt{E}_1]\!]\sigma \in \mathbb{B},\ v_2 = \mathscr{E}[\![\mathtt{E}_2]\!]\sigma \in \mathbb{B} \\
\textit{undefined} & & & \textit{otherwise}
\end{array}
$$

[1] We use a dependent type to express that we require the expression $\mathtt{E}$ to have all of its variables considered in the (implicit) set of variables $\mathbb{V}$. A more detailed discussion on this matter can be found in [Cousot, 2021].

Denotational semantics: expressions (example)

The expression "`(x-1) < (y-1) nand ff`" under $\sigma = \{\mathtt{x} \mapsto 3, \mathtt{y} \mapsto 2\}$ evaluates to:

$$
\begin{aligned}
\mathscr{E}[\![\,\mathtt{(x-1)\ <\ (y-1)\ nand\ ff}\,]\!]\sigma &= \mathscr{E}[\![\,\mathtt{(x-1)\ <\ (y-1)}\,]\!]\sigma \uparrow \mathscr{E}[\![\,\mathtt{ff}\,]\!]\sigma \\
&= \mathscr{E}[\![\,\mathtt{(x-1)}\,]\!]\sigma < \mathscr{E}[\![\,\mathtt{(y-1)}\,]\!]\sigma \uparrow \mathscr{E}[\![\,\mathtt{ff}\,]\!]\sigma \\
&= (\mathscr{E}[\![\,\mathtt{x}\,]\!]\sigma - \mathscr{E}[\![\,1\,]\!]\sigma) < \mathscr{E}[\![\,\mathtt{y-1}\,]\!]\sigma \uparrow \mathscr{E}[\![\,\mathtt{ff}\,]\!]\sigma \\
&= (\mathscr{E}[\![\,\mathtt{x}\,]\!]\sigma - \mathscr{E}[\![\,1\,]\!]\sigma) < (\mathscr{E}[\![\,\mathtt{y}\,]\!]\sigma - \mathscr{E}[\![\,1\,]\!]\sigma) \uparrow \mathscr{E}[\![\,\mathtt{ff}\,]\!]\sigma \\
&= (3-1) < (2-1) \uparrow \mathfrak{ff} \\
&= 2 < 1 \uparrow \mathfrak{ff} \\
&= \mathfrak{ff} \uparrow \mathfrak{ff} \\
&= \mathfrak{tt}
\end{aligned}
$$

Denotational semantics: statements

- Now, we turn our heads to statements.

Denotational semantics of statements

$\mathcal{S} : \mathbb{V} \to (\mathsf{S} \in \mathbb{S}) \to \mathbb{E}_{\mathbb{V}} \rightharpoonup \mathbb{E}_{\mathbb{V}}$ *where* $\mathtt{vars}(\mathsf{S}) \subseteq \mathbb{V}$

Denotational semantics: statements

- Now, we turn our heads to statements.

Denotational semantics of statements

$\mathcal{S} : \mathbb{V} \to (\mathsf{S} \in \mathbb{S}) \to \mathbb{E}\mathbb{v} \rightharpoonup \mathbb{E}\mathbb{v}$ *where* $\mathbb{v}\mathbb{a}\mathbb{r}\mathbb{s}(\mathsf{S}) \subseteq \mathbb{V}$

$$
\begin{array}{lcll}
\mathcal{S}[\![\,\texttt{skip}\,]\!]\sigma & \triangleq & \sigma & \\
\mathcal{S}[\![\,\texttt{x := E}\,]\!]\sigma & \triangleq & \sigma[v/\mathtt{x}] & \text{if } \mathcal{E}[\![\,\mathsf{E}\,]\!]\sigma = v \\
\mathcal{S}[\![\,\mathsf{S}_1\texttt{;}\ \mathsf{S}_2\,]\!]\sigma & \triangleq & (\mathcal{S}[\![\,\mathsf{S}_2\,]\!] \circ \mathcal{S}[\![\,\mathsf{S}_1\,]\!])\sigma &
\end{array}
$$

$$
\mathcal{S}[\![\,\textbf{if}\ (\mathsf{E})\ \textbf{then}\ \mathsf{S}_1\ \textbf{else}\ \mathsf{S}_2\,]\!]\sigma \triangleq \begin{cases} \mathcal{S}[\![\,\mathsf{S}_1\,]\!]\sigma & \text{if } \mathcal{E}[\![\,\mathsf{E}\,]\!]\sigma = \mathfrak{tt} \\ \mathcal{S}[\![\,\mathsf{S}_2\,]\!]\sigma & \text{if } \mathcal{E}[\![\,\mathsf{E}\,]\!]\sigma = \mathfrak{ff} \\ \textit{undefined} & \textit{otherwise} \end{cases}
$$

Denotational semantics: loops

$$\mathcal{S}[\![\texttt{\textbf{while} (E) \textbf{do} S}]\!]\sigma \triangleq \begin{cases} \sigma & \text{if } \mathcal{E}[\![\texttt{E}]\!]\sigma = \mathrm{ff} \\ (\mathcal{S}[\![\texttt{\textbf{while} (E) \textbf{do} S}]\!] \circ \mathcal{S}[\![\texttt{S}]\!])\sigma & \text{if } \mathcal{E}[\![\texttt{E}]\!]\sigma = \mathrm{tt} \\ \textit{undefined} & \textit{otherwise} \end{cases}$$

Denotational semantics: loops

$$\mathcal{S}[\![\texttt{while (E) do S}]\!]\sigma \triangleq \begin{cases} \sigma & \text{if } \mathcal{E}[\![\texttt{E}]\!]\sigma = \mathrm{ff} \\ (\mathcal{S}[\![\texttt{while (E) do S}]\!] \circ \mathcal{S}[\![\texttt{S}]\!])\sigma & \text{if } \mathcal{E}[\![\texttt{E}]\!]\sigma = \mathrm{tt} \\ \textit{undefined} & \textit{otherwise} \end{cases}$$

- *Recursive* definition!

Denotational semantics: loops

$$\mathcal{S}[\![\texttt{while (E) do S}]\!]\sigma \triangleq \begin{cases} \sigma & \text{if } \mathcal{E}[\![\texttt{E}]\!]\sigma = \mathrm{ff} \\ (\mathcal{S}[\![\texttt{while (E) do S}]\!] \circ \mathcal{S}[\![\texttt{S}]\!])\sigma & \text{if } \mathcal{E}[\![\texttt{E}]\!]\sigma = \mathrm{tt} \\ \textit{undefined} & \textit{otherwise} \end{cases}$$

- *Recursive* definition!
- We need to make sure that:
  1. The equation above has (at least) one solution.
  2. If the equation has a solution, we must take the *most precise* one.

Denotational semantics: loops

$$\mathcal{S}[\![\texttt{while (E) do S}]\!]\sigma \triangleq \begin{cases} \sigma & \text{if } \mathcal{E}[\![\texttt{E}]\!]\sigma = \mathrm{ff} \\ (\mathcal{S}[\![\texttt{while (E) do S}]\!] \circ \mathcal{S}[\![\texttt{S}]\!])\sigma & \text{if } \mathcal{E}[\![\texttt{E}]\!]\sigma = \mathrm{tt} \\ \textit{undefined} & \textit{otherwise} \end{cases}$$

- *Recursive* definition!
- We need to make sure that:
  1. The equation above has (at least) one solution.
  2. If the equation has a solution, we must take the *most precise* one.
- And, for this, we use *fixpoints* on *posets* with *information orders*.

Posets & fixpoints: recap

- We start by defining a poset $(\mathscr{I}, \leq_{\mathscr{I}})$ with:

$$\mathscr{I} \triangleq \mathbb{I}_{\perp}, \qquad \text{(values + undefined: } \mathbb{I} \cup \perp\text{)}$$

$$x \leq_{\mathscr{I}} y \triangleq x = \perp \vee x = y. \qquad \text{(information order on values)}$$

That is, we add a bottom-most element $\perp$, the *undefined* value and we define a *flat order*, yielding a nicely structured poset: directed-complete, and finite chains.

$$\begin{tikzcd}
1 \arrow[dr, dash] & 2 \arrow[dr, dash] & \cdots \arrow[dl, dash] & \mathfrak{tt} \arrow[dl, dash] & \mathfrak{ff} \arrow[dll, dash] \\
& & \perp &
\end{tikzcd}$$

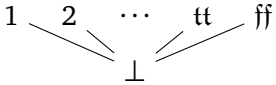
Posets & fixpoints: recap

- We start by defining a poset $(\mathscr{I}, \leq_{\mathscr{I}})$ with:

$$\mathscr{I} \triangleq \mathbb{I}_{\perp}, \qquad \text{(values + undefined: } \mathbb{I} \cup \perp\text{)}$$

$$x \leq_{\mathscr{I}} y \triangleq x = \perp \vee x = y. \qquad \text{(information order on values)}$$

That is, we add a bottom-most element $\perp$, the *undefined* value and we define a *flat order*, yielding a nicely structured poset: directed-complete, and finite chains.



- Similarly, we define the poset $(\mathscr{T}, \leq_{\mathscr{T}})$ with:

$$\mathscr{T} \triangleq \mathbb{E}\mathbb{v}_{\perp} \to \mathbb{E}\mathbb{v}_{\perp}, \qquad \text{(total, continuous state transformers)}$$

$$f \leq_{\mathscr{T}} g \triangleq \forall \sigma : f(\sigma) \dot{\leq}_{\mathscr{I}} g(\sigma). \qquad \text{(information order on state transformers)}$$

Where $\dot{\leq}_{\mathscr{I}}$ is the *point-wise extension* of $\leq_{\mathscr{I}}$ to states, and $\boldsymbol{\lambda}\sigma.\ \perp$ is the bottom-most element. This poset preserves all the nice properties of the one above.

Posets & fixpoints: recap (cont'd)

- In out context:
- the poset $(\mathscr{T}, \leq_{\mathscr{T}})$ is a *directed-complete partial order* (dcpo) since all of its directed subsets have a supremum. That is, $\forall D \subseteq \mathscr{T},\ \textit{directed} : \vee_{\mathscr{T}} D \in \mathscr{T}$; and,

Posets & fixpoints: recap (cont'd)

- In out context:
- the poset $(\mathscr{T}, \leq_{\mathscr{T}})$ is a *directed-complete partial order* (dcpo) since all of its directed subsets have a supremum. That is, $\forall D \subseteq \mathscr{T},\ \textit{directed} : \vee_{\mathscr{T}} D \in \mathscr{T}$; and,
- a function F is *Scott-continuous* in a poset $(\mathscr{T}, \leq_{\mathscr{T}})$ if it preserves all directed suprema. That is, $\forall D \subseteq \mathscr{T},\ \textit{directed} : F(\vee_{\mathscr{T}} D) = \vee_{\mathscr{T}} \{F(f) \mid f \in D\}$.

Posets & fixpoints: recap (cont'd)

- In out context:
- the poset $(\mathscr{T}, \leq_{\mathscr{T}})$ is a *directed-complete partial order* (dcpo) since all of its directed subsets have a supremum. That is, $\forall D \subseteq \mathscr{T},\ \textit{directed} : \vee_{\mathscr{T}} D \in \mathscr{T}$; and,
- a function F is *Scott-continuous* in a poset $(\mathscr{T}, \leq_{\mathscr{T}})$ if it preserves all directed suprema. That is, $\forall D \subseteq \mathscr{T},\ \textit{directed} : F(\vee_{\mathscr{T}} D) = \vee_{\mathscr{T}} \{F(f) \mid f \in D\}$.

Theorem (Kleene's theorem)

Given a directed-complete partial order *(dcpo)* $(\mathscr{T}, \leq_{\mathscr{T}})$*, and a* Scott-continuous function $F : \mathscr{T} \to \mathscr{T}$*, f has a* least fixpoint*, which is the supremum:*

$$\mathbf{lfp}_{\leq_{\mathscr{T}}} F = \vee_{\mathscr{T}} \{F^n(\boldsymbol{\lambda}\sigma.\ \bot) \mid n < \omega\}$$

of the ascending Kleene chain:

$$F^0(\boldsymbol{\lambda}\sigma.\ \bot) \leq_{\mathscr{T}} F^1(\boldsymbol{\lambda}\sigma.\ \bot) \leq_{\mathscr{T}} F^2(\boldsymbol{\lambda}\sigma.\ \bot) \leq_{\mathscr{T}} \ldots \leq_{\mathscr{T}} F^n(\boldsymbol{\lambda}\sigma.\ \bot) \leq_{\mathscr{T}} \ldots$$

Posets & fixpoints: recap (cont'd)

- In out context:
- the poset $(\mathscr{T}, \leq_{\mathscr{T}})$ is a *directed-complete partial order* (dcpo) since all of its directed subsets have a supremum. That is, $\forall D \subseteq \mathscr{T},\ \textit{directed} : \vee_{\mathscr{T}} D \in \mathscr{T}$; and,
- a function F is *Scott-continuous* in a poset $(\mathscr{T}, \leq_{\mathscr{T}})$ if it preserves all directed suprema. That is, $\forall D \subseteq \mathscr{T},\ \textit{directed} : F(\vee_{\mathscr{T}} D) = \vee_{\mathscr{T}} \{F(f) \mid f \in D\}$.

Theorem (Kleene's theorem)

Given a directed-complete partial order *(dcpo)* $(\mathscr{T}, \leq_{\mathscr{T}})$*, and a* Scott-continuous function $F : \mathscr{T} \to \mathscr{T}$*, f has a* least fixpoint*, which is the supremum:*

$$\mathbf{lfp}_{\leq_{\mathscr{T}}} F = \vee_{\mathscr{T}} \{F^n(\boldsymbol{\lambda}\sigma.\ \bot) \mid n < \omega\}$$

of the ascending Kleene chain:

$$F^0(\boldsymbol{\lambda}\sigma.\ \bot) \leq_{\mathscr{T}} F^1(\boldsymbol{\lambda}\sigma.\ \bot) \leq_{\mathscr{T}} F^2(\boldsymbol{\lambda}\sigma.\ \bot) \leq_{\mathscr{T}} \ldots \leq_{\mathscr{T}} F^n(\boldsymbol{\lambda}\sigma.\ \bot) \leq_{\mathscr{T}} \ldots$$

So, our goal now is to make our denotational semantics function $\mathscr{S}$ to "fit" in this setting.

Loop fixpoint semantics

1. Extend the definitions of $\mathscr{E}$ and $\mathscr{S}$ with $\perp$ in the co-domain to model "undefined" behavior and turn them into *total functions*:

$$\mathscr{S} : \mathbb{V} \to (\mathsf{S} \in \mathbb{S}) \to \mathbb{E}\mathtt{v} \to \mathbb{E}\mathtt{v}_{\perp} \textit{ where } \mathtt{vars}(\mathsf{S}) \subseteq \mathbb{V}.$$

Loop fixpoint semantics

1. Extend the definitions of $\mathscr{E}$ and $\mathscr{S}$ with $\bot$ in the co-domain to model "undefined" behavior and turn them into *total functions*:

$$\mathscr{S} : \mathbb{V} \to (\mathsf{S} \in \mathbb{S}) \to \mathbb{E}\mathtt{v} \to \mathbb{E}\mathtt{v}_\bot \textit{ where } \mathtt{vars}(\mathsf{S}) \subseteq \mathbb{V}.$$

2. Make them composable (*i.e.*, domain = co-domain) by allowing $\bot$ in the domain:

$$\mathscr{S} : \mathbb{V} \to (\mathsf{S} \in \mathbb{S}) \to \mathbb{E}\mathtt{v}_\bot \to \mathbb{E}\mathtt{v}_\bot \textit{ where } \mathtt{vars}(\mathsf{S}) \subseteq \mathbb{V}.$$

Loop fixpoint semantics

1. Extend the definitions of $\mathscr{E}$ and $\mathscr{S}$ with $\bot$ in the co-domain to model "undefined" behavior and turn them into *total functions*:

$$\mathscr{S} : \mathbb{V} \to (\mathsf{S} \in \mathbb{S}) \to \mathbb{E}\mathbb{v} \to \mathbb{E}\mathbb{v}_\bot \textit{ where } \mathtt{vars}(\mathsf{S}) \subseteq \mathbb{V}.$$

2. Make them composable (*i.e.*, domain = co-domain) by allowing $\bot$ in the domain:

$$\mathscr{S} : \mathbb{V} \to (\mathsf{S} \in \mathbb{S}) \to \mathbb{E}\mathbb{v}_\bot \to \mathbb{E}\mathbb{v}_\bot \textit{ where } \mathtt{vars}(\mathsf{S}) \subseteq \mathbb{V}.$$

3. Prove that $\mathscr{S}[\![_]\!] : \mathbb{E}\mathbb{v}_\bot \to \mathbb{E}\mathbb{v}_\bot$ is *Scott-continuous*.
   (Sketch: by structural induction on the program's syntax)

Loop fixpoint semantics (cont'd)

Now, we can express the solution to the **`while`**'s (semantic) equation as the *least fixpoint* (*i.e.*, most-precise for the information order) of the *functional* F:

$$\mathcal{S}[\![\texttt{while (E) do S}]\!]\sigma \;\triangleq\; \mathit{lfp}_{\leq_{\mathcal{T}}} F$$

where $F : (\mathbb{E}\mathbb{v}_\perp \to \mathbb{E}\mathbb{v}_\perp) \to (\mathbb{E}\mathbb{v}_\perp \to \mathbb{E}\mathbb{v}_\perp)$, (or, $F : \mathcal{T} \to \mathcal{T}$)

$$F(f)(\sigma) = \begin{cases} \sigma & \text{if } \mathcal{E}[\![\texttt{E}]\!]\sigma = \mathfrak{ff} \\ (f \circ \mathcal{S}[\![\texttt{S}]\!])\sigma & \text{if } \mathcal{E}[\![\texttt{E}]\!]\sigma = \mathfrak{tt} \\ \perp & \textit{otherwise} \end{cases}$$

Loop fixpoint semantics: example

- Consider computing the semantics of the program:

```
while (0 < n) do
  x := x - n; n := n - 1
```

- We know that $\mathcal{S}[\![\texttt{x := x - n; n := n - 1}]\!]\sigma = \sigma[x-n/x, n-1/n]$.
- We compute the least fixpoint of the functional F defined as follows:

$$F(f)(\sigma) = \begin{cases} \sigma & \text{if } \mathcal{E}[\![\texttt{0 < n}]\!]\sigma = \mathrm{ff} \\ f \circ \sigma[x-n/x, n-1/n] & \text{if } \mathcal{E}[\![\texttt{0 < n}]\!]\sigma = \mathrm{tt} \\ \bot & \textit{otherwise} \end{cases}$$

by building the Kleene ascending chain $F^0(\boldsymbol{\lambda}\sigma.\,\bot) \leq_{\mathcal{F}} F^1(\boldsymbol{\lambda}\sigma.\,\bot) \leq_{\mathcal{F}} F^2(\boldsymbol{\lambda}\sigma.\,\bot) \leq_{\mathcal{F}} \ldots$

Loop fixpoint semantics: example

$F^0(\boldsymbol{\lambda}\sigma.\,\bot)(\sigma) = \bot$ (no information)

Loop fixpoint semantics: example

$$F^1(\boldsymbol{\lambda}\sigma.\,\bot)(\sigma) = \begin{cases} \sigma & \text{if } \mathscr{E}[\![\,\mathtt{0}\ <\ \mathtt{n}\,]\!]\sigma = \mathfrak{ff} \\ \bot & \textit{otherwise} \end{cases}$$

(at most 0 iterations)

Loop fixpoint semantics: example

$$F^2(\boldsymbol{\lambda}\sigma.\,\bot)(\sigma) = \begin{cases} \mathcal{S}[\![\,\mathtt{S}\,]\!]\sigma & \text{if } (\mathcal{E}[\![\,\mathtt{0}\ <\ \mathtt{n}\,]\!] \circ \mathcal{S}[\![\,\mathtt{S}\,]\!])\sigma = \mathfrak{ff} \\ \sigma & \text{if } \mathcal{E}[\![\,\mathtt{0}\ <\ \mathtt{n}\,]\!]\sigma = \mathfrak{ff} \\ \bot & \textit{otherwise} \end{cases}$$

(at most 1 iteration)

Loop fixpoint semantics: example

$$F^3(\boldsymbol{\lambda}\sigma.\,\bot)(\sigma) = \begin{cases} (\mathcal{S}[\![\,\mathtt{S}\,]\!] \circ \mathcal{S}[\![\,\mathtt{S}\,]\!])\sigma & \text{if } (\mathcal{E}[\![\,\mathtt{0} \ < \ \mathtt{n}\,]\!] \circ \mathcal{S}[\![\,\mathtt{S}\,]\!] \circ \mathcal{S}[\![\,\mathtt{S}\,]\!])\sigma = \mathfrak{ff} \\ \mathcal{S}[\![\,\mathtt{S}\,]\!]\sigma & \text{if } (\mathcal{E}[\![\,\mathtt{0} \ < \ \mathtt{n}\,]\!] \circ \mathcal{S}[\![\,S\,]\!])\sigma = \mathfrak{ff} \\ \sigma & \text{if } \mathcal{E}[\![\,\mathtt{0} \ < \ \mathtt{n}\,]\!]\sigma = \mathfrak{ff} \\ \bot & \textit{otherwise} \end{cases}$$

(at most 2 iterations)

Loop fixpoint semantics: example

$F^n(\boldsymbol{\lambda}\sigma.\,\bot)(\sigma)$ (at most $n-1$ iterations)

Loop fixpoint semantics: example

$\mathit{lfp}_{\leq_{\mathscr{T}}} F = \vee_{\mathscr{T}} \{F^n(\boldsymbol{\lambda}\sigma.\ \bot)\ :\ n < \omega\}$ (any number of iterations)

Properties

Properties as sets

- Given a set of entities (or universe) $\mathscr{U}$, properties are *sets of entities* in $\mathscr{U}$ belonging to $\wp(\mathscr{U})$ that have these properties.

Properties as sets

- Given a set of entities (or universe) $\mathcal{U}$, properties are *sets of entities* in $\mathcal{U}$ belonging to $\wp(\mathcal{U})$ that have these properties.
- An entity $e \in \mathcal{U}$ has a property $P \in \wp(\mathcal{U})$ iff $e \in P$, or equivalently, $\{e\} \subseteq P$ (implication).

Properties as sets

- Given a set of entities (or universe) $\mathcal{U}$, properties are *sets of entities* in $\mathcal{U}$ belonging to $\wp(\mathcal{U})$ that have these properties.
- An entity $e \in \mathcal{U}$ has a property $P \in \wp(\mathcal{U})$ iff $e \in P$, or equivalently, $\{e\} \subseteq P$ (implication).
- $\emptyset$ is the *strongest* property. (false)
- $\mathcal{U}$ is the *weakest* property. (true)
- $\subseteq$ is *implication*. ($\rightarrow$)

Properties as sets

- Given a set of entities (or universe) $\mathcal{U}$, properties are *sets of entities* in $\mathcal{U}$ belonging to $\wp(\mathcal{U})$ that have these properties.
- An entity $e \in \mathcal{U}$ has a property $P \in \wp(\mathcal{U})$ iff $e \in P$, or equivalently, $\{e\} \subseteq P$ (implication).
- $\emptyset$ is the *strongest* property. (false)
- $\mathcal{U}$ is the *weakest* property. (true)
- $\subseteq$ is *implication*. ($\rightarrow$)

Two important remarks:

- Herein, $e \in P \Longleftrightarrow \{e\} \subseteq P$, and, although it may seem weird at first, the latter is the preferred one, since: (**i**) the use of $\subseteq$ allows us to move from set theory to order/lattice theory (monotone functions, fixpoints, ...), and (**ii**) abstracting $\subseteq$ is way easier than $\in$.

Properties as sets

- Given a set of entities (or universe) $\mathcal{U}$, properties are *sets of entities* in $\mathcal{U}$ belonging to $\wp(\mathcal{U})$ that have these properties.
- An entity $e \in \mathcal{U}$ has a property $P \in \wp(\mathcal{U})$ iff $e \in P$, or equivalently, $\{e\} \subseteq P$ (implication).
- $\emptyset$ is the *strongest* property. (false)
- $\mathcal{U}$ is the *weakest* property. (true)
- $\subseteq$ is *implication*. ($\rightarrow$)

Two important remarks:

- Herein, $e \in P \Longleftrightarrow \{e\} \subseteq P$, and, although it may seem weird at first, the latter is the preferred one, since: (**i**) the use of $\subseteq$ allows us to move from set theory to order/lattice theory (monotone functions, fixpoints, ...), and (**ii**) abstracting $\subseteq$ is way easier than $\in$.
- By considering properties as sets, we give raise to a Boolean algebra, since we are dealing with powerset-shaped (complete) lattices.

Properties as sets: example

For example, take our universe $\mathscr{U}$ to be $\mathbb{Z}$, so that properties are in $\wp(\mathbb{Z})$.

- $\emptyset$ is false (no integer has this property).
- $\mathbb{Z}$ is true (all integers have this property).
- $2\mathbb{Z} \triangleq \{z \in \mathbb{Z} \mid \exists k \in \mathbb{Z} : z = 2k\}$ is the even property.
- $2\mathbb{Z} + 1 \triangleq \{z \in \mathbb{Z} \mid \exists k \in \mathbb{Z} : z = 2k + 1\}$ is the odd property.

Intuitively, and integer can only be even or odd, expressed as: $\mathbb{Z} = 2\mathbb{Z} \cup 2\mathbb{Z} + 1$.

Program semantic properties

- With $\mathscr{U} = \mathbb{E}\mathbb{v}_{\perp}$, program semantic properties are elements in $\wp(\mathbb{E}\mathbb{v}_{\perp})$.

Program semantic properties

- With $\mathscr{U} = \mathbb{E}\mathbb{v}_\perp$, program semantic properties are elements in $\wp(\mathbb{E}\mathbb{v}_\perp)$.
- The *strongest* semantic property of a program $\mathtt{P}$ is its *static* or *collecting* semantics $\mathscr{S}^*[\![\mathtt{P}]\!]$.

Program semantic properties

- With $\mathscr{U} = \mathbb{E}_{\mathbb{v}\perp}$, program semantic properties are elements in $\wp(\mathbb{E}_{\mathbb{v}\perp})$.
- The *strongest* semantic property of a program $\mathtt{P}$ is its *static* or *collecting* semantics $\mathscr{S}^*[\![\mathtt{P}]\!]$.
- The definition is a bit cumbersome...

Collecting semantics: expressions

Collecting semantics of expressions

$\mathscr{E} : \mathbb{V} \to (\mathtt{E} \in \mathbb{E}) \to \mathbb{E}_{\mathbb{V}} \to \wp(\mathbb{I}_\perp)$ *where* $\mathtt{vars}(\mathtt{E}) \subseteq \mathbb{V}$

$$
\begin{array}{lcll}
\mathscr{E}[\![\mathtt{k}]\!]\sigma & \triangleq & \{k\} & \in \wp(\mathbb{I}) \\
\mathscr{E}[\![\mathtt{x}]\!]\sigma & \triangleq & \{\sigma(\mathtt{x})\} & \in \wp(\mathbb{I}_\perp) \\
\mathscr{E}[\![\mathtt{E}_1 \ \mathtt{-} \ \mathtt{E}_2]\!]\sigma & \triangleq & \{v_1 - v_2 \mid v_1 \in \mathscr{E}[\![\mathtt{E}_1]\!]\sigma, v_2 \in \mathscr{E}[\![\mathtt{E}_2]\!]\sigma\} & \in \wp(\mathbb{Z}) \\
\mathscr{E}[\![\mathtt{E}_1 \ \mathtt{<} \ \mathtt{E}_2]\!]\sigma & \triangleq & \{v_1 < v_2 \mid v_1 \in \mathscr{E}[\![\mathtt{E}_1]\!]\sigma, v_2 \in \mathscr{E}[\![\mathtt{E}_2]\!]\sigma\} & \in \wp(\mathbb{B}) \\
\mathscr{E}[\![\mathtt{E}_1 \ \mathtt{nand} \ \mathtt{E}_2]\!]\sigma & \triangleq & \{v_1 \uparrow v_2 \mid v_1 \in \mathscr{E}[\![\mathtt{E}_1]\!]\sigma, v_2 \in \mathscr{E}[\![\mathtt{E}_2]\!]\sigma\} & \in \wp(\mathbb{B}) \\
\perp & & & \textit{otherwise}
\end{array}
$$

Collecting semantics: statements

Denotational semantics of statements

$\mathcal{S} : \mathbb{V} \to (\mathtt{S} \in \mathbb{S}) \to \wp(\mathbb{E}\mathrm{v}_\perp) \to \wp(\mathbb{E}\mathrm{v}_\perp)$ *where* $\mathtt{vars}(\mathtt{S}) \subseteq \mathbb{V}$

$$
\begin{array}{lcl}
\mathcal{S}[\![\,\mathtt{skip}\,]\!]\Sigma & \triangleq & \Sigma \\
\mathcal{S}[\![\,\mathtt{x}\ :=\ \mathtt{E}\,]\!]\Sigma & \triangleq & \{\sigma[v/\mathtt{x}] \mid \sigma \in \Sigma, v \in \mathcal{E}[\![\,\mathtt{E}\,]\!]\sigma\} \\
\mathcal{S}[\![\,\mathtt{S}_1;\ \mathtt{S}_2\,]\!]\Sigma & \triangleq & (\mathcal{S}[\![\,\mathtt{S}_2\,]\!] \circ \mathcal{S}[\![\,\mathtt{S}_1\,]\!])\Sigma \\
\mathcal{S}[\![\,\mathbf{if}\ (\mathtt{E})\ \mathbf{then}\ \mathtt{S}_1\ \mathbf{else}\ \mathtt{S}_2\,]\!]\Sigma & \triangleq & \mathcal{S}[\![\,\mathtt{S}_1\,]\!]\{\sigma \in \Sigma \mid \mathrm{tt} \in \mathcal{E}[\![\,\mathtt{E}\,]\!]\sigma\} \\
 & \cup & \mathcal{S}[\![\,\mathtt{S}_2\,]\!]\{\sigma \in \Sigma \mid \mathrm{ff} \in \mathcal{E}[\![\,\mathtt{E}\,]\!]\sigma\} \\
\mathcal{S}[\![\,\mathbf{while}\ (\mathtt{E})\ \mathbf{do}\ \mathtt{S}\,]\!]\Sigma & \triangleq & \mathit{lfp}_{\leq_{\mathcal{F}}} F
\end{array}
$$

with: $F(f)(\Sigma) = \{\sigma \in \Sigma \mid \mathrm{ff} \in \mathcal{E}[\![\,\mathtt{E}\,]\!]\sigma\} \cup (f \circ \mathcal{S}^*[\![\,\mathtt{S}\,]\!])\{\sigma \in \Sigma \mid \mathrm{tt} \in \mathcal{E}[\![\,\mathtt{E}\,]\!]\sigma\}$

$\emptyset$ *otherwise*

Abstract Interpretation

Motivation: undecidability

Proving non-trivial, program semantic properties is UNDECIDABLE in general.

Motivation: undecidability

Proving non-trivial, program semantic properties is UNDECIDABLE in general.

Abstract interpretation formalizes a theory for computing sound approximations of program semantic properties. In general, we (roughly) follow the following steps:

Motivation: undecidability

Proving non-trivial, program semantic properties is UNDECIDABLE in general.

Abstract interpretation formalizes a theory for computing sound approximations of program semantic properties. In general, we (roughly) follow the following steps:

Abstract domains

- Consider a subset of the *concrete domain* (in our case, $(\wp(\mathbb{E}\mathbb{v}_\perp), \subseteq)$ of *properties of interest*, and encode such properties in an *abstract domain* $(\mathfrak{A}, \preccurlyeq)$.

Motivation: undecidability

Proving non-trivial, program semantic properties is UNDECIDABLE in general.

Abstract interpretation formalizes a theory for computing sound approximations of program semantic properties. In general, we (roughly) follow the following steps:

Abstract domains

- Consider a subset of the *concrete domain* (in our case, $(\wp(\mathbb{E}\mathbb{v}_\perp), \subseteq)$ of *properties of interest*, and encode such properties in an *abstract domain* $(\mathfrak{A}, \preccurlyeq)$.

Galois connections & fixpoint abstraction

- Establish a connection between concrete properties $P \in \wp(\mathbb{E}\mathbb{v}_\perp)$ and abstract properties $P^\sharp \in \mathfrak{A}$ such that proofs in the abstract *imply* proofs in the concrete.

Motivation: undecidability

Proving non-trivial, program semantic properties is UNDECIDABLE in general.

Abstract interpretation formalizes a theory for computing sound approximations of program semantic properties. In general, we (roughly) follow the following steps:

Abstract domains

- Consider a subset of the *concrete domain* (in our case, $(\wp(\mathbb{E}\mathbb{v}_\perp), \subseteq)$ of *properties of interest*, and encode such properties in an *abstract domain* $(\mathfrak{A}, \preccurlyeq)$.

Galois connections & fixpoint abstraction

- Establish a connection between concrete properties $P \in \wp(\mathbb{E}\mathbb{v}_\perp)$ and abstract properties $P^\sharp \in \mathfrak{A}$ such that proofs in the abstract *imply* proofs in the concrete.

Transformer abstraction

- Design an abstract semantic transformer $\mathcal{S}^\sharp[\![\mathtt{P}]\!]$ that over-approximates the concrete (collecting) semantic transformer $\mathcal{S}^*[\![\mathtt{P}]\!]$.

Abstract domains: interval abstraction

- We'll design an abstract domain for *interval analysis*.
- Useful for, *e.g.*, detecting out-of-bound array accesses.

```
n := 1000;
```
$\{n \in [1000, 1000]\}$
```
while (0 < n) do
```
$\{n \in [1, 1000]\}$
```
  n := n - 1
```
$\{n \in [0, 999]\}$

$\{n \in [0, 0]\}$

Interval abstraction: the lattice

- The set $\mathfrak{I}$ is defined as: $\mathfrak{I} \triangleq \{\bot\} \cup \{[\ell, r] \mid \ell \in \mathbb{Z}^{-\infty}, r \in \mathbb{Z}^{+\infty}, \ell \leq r\}$.

Interval abstraction: the lattice

- The set $\mathfrak{I}$ is defined as: $\mathfrak{I} \triangleq \{\bot\} \cup \{[\ell, r] \mid \ell \in \mathbb{Z}^{-\infty}, r \in \mathbb{Z}^{+\infty}, \ell \leq r\}$.
- its order relation $\preccurlyeq$ as: $[\ell_1, r_1] \preccurlyeq [\ell_2, r_2] \Leftrightarrow \ell_2 \leq \ell_1 \wedge r_1 \leq r_2$.

Interval abstraction: the lattice

- The set $\mathfrak{I}$ is defined as: $\mathfrak{I} \triangleq \{\bot\} \cup \{[\ell, r] \mid \ell \in \mathbb{Z}^{-\infty}, r \in \mathbb{Z}^{+\infty}, \ell \leq r\}$.
- its order relation $\preccurlyeq$ as: $[\ell_1, r_1] \preccurlyeq [\ell_2, r_2] \Leftrightarrow \ell_2 \leq \ell_1 \wedge r_1 \leq r_2$.
- its *lub* $\curlyvee$ as: $[\ell_1, r_1] \curlyvee [\ell_2, r_2] \triangleq [min\{\ell_1, \ell_2\}, max\{r_1, r_2\}]$, with $\bot$ being the neutral element.

Interval abstraction: the lattice

- The set $\mathfrak{I}$ is defined as: $\mathfrak{I} \triangleq \{\bot\} \cup \{[\ell, r] \mid \ell \in \mathbb{Z}^{-\infty}, r \in \mathbb{Z}^{+\infty}, \ell \leq r\}$.
- its order relation $\preccurlyeq$ as: $[\ell_1, r_1] \preccurlyeq [\ell_2, r_2] \Leftrightarrow \ell_2 \leq \ell_1 \wedge r_1 \leq r_2$.
- its *lub* $\curlyvee$ as: $[\ell_1, r_1] \curlyvee [\ell_2, r_2] \triangleq [min\{\ell_1, \ell_2\}, max\{r_1, r_2\}]$, with $\bot$ being the neutral element.
- and its *glb* $\curlywedge$ as: $[\ell_1, r_1] \curlywedge [\ell_2, r_2] \triangleq \begin{cases} \bot & \textit{if } r_1 < \ell_2 \vee r_2 < \ell_1 \\ [max\{\ell_1, \ell_2\}, min\{r_1, r_2\}] & \textit{otherwise} \end{cases}$, with $\bot$ being the absorbing element.

Interval abstraction: the lattice

- The set $\mathfrak{I}$ is defined as: $\mathfrak{I} \triangleq \{\bot\} \cup \{[\ell, r] \mid \ell \in \mathbb{Z}^{-\infty}, r \in \mathbb{Z}^{+\infty}, \ell \leq r\}$.
- its order relation $\preccurlyeq$ as: $[\ell_1, r_1] \preccurlyeq [\ell_2, r_2] \Leftrightarrow \ell_2 \leq \ell_1 \wedge r_1 \leq r_2$.
- its *lub* $\curlyvee$ as: $[\ell_1, r_1] \curlyvee [\ell_2, r_2] \triangleq [min\{\ell_1, \ell_2\}, max\{r_1, r_2\}]$, with $\bot$ being the neutral element.
- and its *glb* $\curlywedge$ as: $[\ell_1, r_1] \curlywedge [\ell_2, r_2] \triangleq \begin{cases} \bot & \textit{if } r_1 < \ell_2 \vee r_2 < \ell_1 \\ [max\{\ell_1, \ell_2\}, min\{r_1, r_2\}] & \textit{otherwise} \end{cases}$, with $\bot$ being the absorbing element.
- Under this setting, $(\mathfrak{I}, \preccurlyeq)$ is a *complete* lattice.

Interval abstraction: the lattice

- The set $\mathfrak{I}$ is defined as: $\mathfrak{I} \triangleq \{\bot\} \cup \{[\ell, r] \mid \ell \in \mathbb{Z}^{-\infty}, r \in \mathbb{Z}^{+\infty}, \ell \leq r\}$.
- its order relation $\preccurlyeq$ as: $[\ell_1, r_1] \preccurlyeq [\ell_2, r_2] \Leftrightarrow \ell_2 \leq \ell_1 \wedge r_1 \leq r_2$.
- its *lub* $\curlyvee$ as: $[\ell_1, r_1] \curlyvee [\ell_2, r_2] \triangleq [min\{\ell_1, \ell_2\}, max\{r_1, r_2\}]$, with $\bot$ being the neutral element.
- and its *glb* $\curlywedge$ as: $[\ell_1, r_1] \curlywedge [\ell_2, r_2] \triangleq \begin{cases} \bot & \textit{if } r_1 < \ell_2 \vee r_2 < \ell_1 \\ [max\{\ell_1, \ell_2\}, min\{r_1, r_2\}] & \textit{otherwise} \end{cases}$, with $\bot$ being the absorbing element.
- Under this setting, $(\mathfrak{I}, \preccurlyeq)$ is a *complete* lattice.
- And we point-wise extend it to $(\mathbb{E}\mathbb{v}^\sharp, \dot{\preccurlyeq})$ where $\mathbb{E}\mathbb{v}^\sharp \triangleq \mathbb{V} \to \mathfrak{I}$, and the relations/operators are adapted acoordingly, while preserving a complete lattice.

Interval abstraction: minus operation

- The interval minus operation $-^\sharp : \mathfrak{I} \times \mathfrak{I} \to \mathfrak{I}$ defines the interval $[\ell_1, r_1] -^\sharp [\ell_2, r_2]$ of `x - y` when `x` has the interval $[\ell_1, r_1]$, and `y` has the interval $[\ell_2, r_2]$.

Interval abstraction: minus operation

- The interval minus operation $-^\sharp : \mathfrak{I} \times \mathfrak{I} \to \mathfrak{I}$ defines the interval $[\ell_1, r_1] -^\sharp [\ell_2, r_2]$ of `x - y` when `x` has the interval $[\ell_1, r_1]$, and `y` has the interval $[\ell_2, r_2]$.
- It is defined as follows:

$$[\ell_1, r_1] -^\sharp [\ell_2, r_2] \triangleq [\ell_1 - r_2, r_1 - \ell_2]$$

Interval abstraction: abstract semantics of expressions

Abstract semantics of expressions

$\mathscr{E}^\sharp : \mathbb{V} \to (\mathtt{E} \in \mathbb{E}) \to \mathbb{E}\mathbb{v}^\sharp \to \mathfrak{I}$ *where* $\mathtt{vars}(\mathtt{E}) \subseteq \mathbb{V}$

$$
\begin{array}{lcll}
\mathscr{E}^\sharp[\![\mathtt{E}]\!]\sigma^\sharp & \triangleq & \bot & \in \mathfrak{I} \quad \text{if some variable in } \sigma^\sharp \text{ has the interval } \bot \\
\mathscr{E}^\sharp[\![\mathtt{1}]\!]\sigma^\sharp & \triangleq & [1,1] & \in \mathfrak{I} \\
\mathscr{E}^\sharp[\![\mathtt{tt}]\!]\sigma^\sharp & \triangleq & [1,1] & \in \mathfrak{I} \\
\mathscr{E}^\sharp[\![\mathtt{ff}]\!]\sigma^\sharp & \triangleq & [0,0] & \in \mathfrak{I} \\
\mathscr{E}^\sharp[\![\mathtt{x}]\!]\sigma^\sharp & \triangleq & \sigma^\sharp(\mathtt{x}) & \in \mathfrak{I} \\
\mathscr{E}^\sharp[\![\mathtt{E}_1 \ \mathtt{-} \ \mathtt{E}_2]\!]\sigma^\sharp & \triangleq & \mathscr{E}^\sharp[\![\mathtt{E}_1]\!]\sigma^\sharp -^\sharp \mathscr{E}^\sharp[\![\mathtt{E}_2]\!]\sigma^\sharp & \in \mathfrak{I} \\
\mathscr{E}^\sharp[\![\mathtt{E}_1 \ \mathtt{<} \ \mathtt{E}_2]\!]\sigma^\sharp & \triangleq & \mathscr{E}^\sharp[\![\mathtt{E}_1]\!]\sigma^\sharp \prec \mathscr{E}^\sharp[\![\mathtt{E}_2]\!]\sigma^\sharp & \in \{[0,0],[1,1],[0,1]\} \\
\bot & & & \textit{otherwise}
\end{array}
$$

Interval abstraction: abstract semantics of statements

Abstract semantics of statements

$\mathcal{S}^\sharp : \mathbb{V} \to (\mathtt{E} \in \mathbb{E}) \to \mathbb{E}\mathrm{v}^\sharp \to \mathbb{E}\mathrm{v}^\sharp$ *where* $\mathtt{vars}(\mathtt{E}) \subseteq \mathbb{V}$

$$\mathcal{S}^\sharp[\![\mathtt{skip}]\!]\sigma^\sharp \triangleq \sigma^\sharp$$

$$\mathcal{S}^\sharp[\![\mathtt{x} := \mathtt{E}]\!]\sigma^\sharp \triangleq \sigma^\sharp[\mathcal{E}^\sharp[\![\mathtt{E}]\!]\sigma^\sharp/\mathtt{x}]$$

$$\mathcal{S}^\sharp[\![\mathtt{S}_1;\ \mathtt{S}_2]\!]\sigma^\sharp \triangleq (\mathcal{S}^\sharp[\![\mathtt{S}_2]\!] \circ \mathcal{S}^\sharp[\![\mathtt{S}_1]\!])\sigma^\sharp$$

Interval abstraction: abstract semantics of statements

$$\mathcal{S}^\sharp[\![\texttt{if (E) then } \mathrm{S}_1 \texttt{ else } \mathrm{S}_2]\!]\sigma^\sharp \quad \triangleq \quad \sigma_1^\sharp \curlyvee \sigma_2^\sharp \textit{ with}$$

$$\sigma_1^\sharp = \begin{cases} \mathcal{S}^\sharp[\![\mathrm{S}_1]\!]\sigma^\sharp & \text{if } 1 \in \mathcal{E}^\sharp[\![E]\!]\sigma^\sharp \\ \bot & \text{otherwise} \end{cases}$$

$$\sigma_2^\sharp = \begin{cases} \mathcal{S}^\sharp[\![\mathrm{S}_2]\!]\sigma^\sharp & \text{if } 0 \in \mathcal{E}^\sharp[\![E]\!]\sigma^\sharp \\ \bot & \text{otherwise} \end{cases}$$

Interval abstraction: abstract semantics of statements

$$\mathcal{S}^\sharp[\![\texttt{while (E) do S}]\!]\sigma^\sharp \quad \triangleq \quad \mathit{lfp}_{\dot{\preccurlyeq}} F^\sharp \textit{ with}$$

$$F^\sharp(f^\sharp)(\sigma^\sharp) = \sigma^\sharp \curlyvee \begin{cases} (f^\sharp \circ \mathcal{S}^\sharp[\![\,\mathrm{S}\,]\!])\sigma^\sharp & \text{if } 1 \in \mathcal{E}^\sharp[\![E]\!]\sigma^\sharp \\ \bot & \text{otherwise} \end{cases}$$

Interval abstraction: soundness

- We would like to prove that, for a given set of initial states Σ, the *interval* semantics $\mathcal{S}^\sharp[\![\,\mathrm{P}\,]\!]$ is a *weaker property* than the *collecting* semantics $\mathcal{S}^*[\![\,\mathrm{P}\,]\!]$.

Interval abstraction: soundness

- We would like to prove that, for a given set of initial states Σ, the *interval* semantics $\mathcal{S}^\sharp[\![\,\mathtt{P}\,]\!]$ is a *weaker property* than the *collecting* semantics $\mathcal{S}^*[\![\,\mathtt{P}\,]\!]$.
- For that, we need to map abstract properties in $\mathbb{E}\mathbb{v}^\sharp$ to concrete properties in $\wp(\mathbb{E}\mathbb{v}_\perp)$, via the *concretization* function

$$\dot{\gamma} : \mathbb{E}\mathbb{v}^\sharp \to \wp(\mathbb{E}\mathbb{v}_\perp),$$

  and vice-versa, via the *abstraction* function

$$\dot{\alpha} : \wp(\mathbb{E}\mathbb{v}_\perp) \to \mathbb{E}\mathbb{v}^\sharp.$$

Interval abstraction: soundness

- We would like to prove that, for a given set of initial states Σ, the *interval* semantics $\mathcal{S}^\sharp[\![\,\mathtt{P}\,]\!]$ is a *weaker property* than the *collecting* semantics $\mathcal{S}^*[\![\,\mathtt{P}\,]\!]$.
- For that, we need to map abstract properties in $\mathbb{E}\mathbb{v}^\sharp$ to concrete properties in $\wp(\mathbb{E}\mathbb{v}_\perp)$, via the *concretization* function

$$\dot{\gamma} : \mathbb{E}\mathbb{v}^\sharp \to \wp(\mathbb{E}\mathbb{v}_\perp),$$

  and vice-versa, via the *abstraction* function

$$\dot{\alpha} : \wp(\mathbb{E}\mathbb{v}_\perp) \to \mathbb{E}\mathbb{v}^\sharp.$$

- And they should yield *equivalent* counterparts.

Interval abstraction: soundness

- We would like to prove that, for a given set of initial states Σ, the *interval* semantics $\mathcal{S}^\sharp[\![\,\mathrm{P}\,]\!]$ is a *weaker property* than the *collecting* semantics $\mathcal{S}^*[\![\,\mathrm{P}\,]\!]$.
- For that, we need to map abstract properties in $\mathbb{E}\mathbb{v}^\sharp$ to concrete properties in $\wp(\mathbb{E}\mathbb{v}_\perp)$, via the *concretization* function

$$\dot{\gamma} : \mathbb{E}\mathbb{v}^\sharp \to \wp(\mathbb{E}\mathbb{v}_\perp),$$

  and vice-versa, via the *abstraction* function

$$\dot{\alpha} : \wp(\mathbb{E}\mathbb{v}_\perp) \to \mathbb{E}\mathbb{v}^\sharp.$$

- And they should yield *equivalent* counterparts.
- Formally,

$$\mathcal{S}^*[\![\,\mathrm{P}\,]\!]\Sigma \subseteq (\dot{\gamma} \circ \mathcal{S}^\sharp[\![\,\mathrm{P}\,]\!] \circ \dot{\alpha})\Sigma.$$

Interval abstraction: concretization

We define $\dot{\gamma}$ in steps. First, $\gamma : \mathfrak{I} \to \wp(\mathbb{I}_\perp)$, where:

$$\gamma(\perp) \triangleq \emptyset$$
$$\gamma([\ell, r]) \triangleq \{z \in \mathbb{Z} \mid \ell \leq z \leq r\}$$

Then, we point-wise extend $\dot{\gamma} : \mathbb{E}\mathbb{v}^\sharp \to \wp(\mathbb{E}\mathbb{v}_\perp)$, where:

$$\dot{\gamma}(\sigma^\sharp) \triangleq \{\sigma \in \mathbb{E}\mathbb{v}_\perp \mid \forall \mathtt{x} \in \mathbb{V} \ : \ \sigma(\mathtt{x}) \in \gamma(\sigma^\sharp(\mathtt{x}))\}$$

Interval abstraction: abstraction

We define $\dot{\alpha}$ in steps. First, $\alpha : \wp(\mathbb{I}_\perp) \to \mathfrak{I}$, where:

$$\alpha(\emptyset) \triangleq \perp$$
$$\alpha(Z) \triangleq [min\, Z, max\, Z]$$

Then, we point-wise extend $\dot{\alpha} : \wp(\mathbb{E}\mathbb{v}_\perp) \to \mathbb{E}\mathbb{v}^\sharp$, where:

$$\dot{\alpha}(\Sigma) \triangleq \{\mathtt{x} \mapsto [min\{\sigma(\mathtt{x}) \mid \sigma \in \Sigma\}, max\{\sigma(\mathtt{x}) \mid \sigma \in \Sigma\}] \mid \mathtt{x} \in \mathbb{V}\}$$

Interval abstraction: Galois connection

- The abstraction/concretization functions $(\dot{\alpha}, \dot{\gamma})$ are closely related in that, for every $P \in \wp(\mathbb{E}\mathbb{v}_\perp)$ and $P^\sharp \in \mathbb{E}\mathbb{v}^\sharp$,

$$\dot{\alpha}(P) \dot{\preccurlyeq} P^\sharp \iff P \subseteq \dot{\gamma}(P^\sharp).$$

Interval abstraction: Galois connection

- The abstraction/concretization functions $(\dot{\alpha}, \dot{\gamma})$ are closely related in that, for every $P \in \wp(\mathbb{E}\mathbb{v}_\perp)$ and $P^\sharp \in \mathbb{E}\mathbb{v}^\sharp$,
$$\dot{\alpha}(P) \dot{\preccurlyeq} P^\sharp \Longleftrightarrow P \subseteq \dot{\gamma}(P^\sharp).$$

- Which is the definition of a Galois connection (GC), which we write:
$$(\wp(\mathbb{E}\mathbb{v}_\perp), \subseteq) \underset{\dot{\alpha}}{\overset{\dot{\gamma}}{\leftrightarrows}} (\mathbb{E}\mathbb{v}^\sharp, \dot{\preccurlyeq})$$

Interval abstraction: Galois connection

- The abstraction/concretization functions $(\dot{\alpha}, \dot{\gamma})$ are closely related in that, for every $P \in \wp(\mathbb{E}\mathbb{v}_\perp)$ and $P^\sharp \in \mathbb{E}\mathbb{v}^\sharp$,
$$\dot{\alpha}(P) \dot{\preccurlyeq} P^\sharp \Longleftrightarrow P \subseteq \dot{\gamma}(P^\sharp).$$

- Which is the definition of a Galois connection (GC), which we write:
$$(\wp(\mathbb{E}\mathbb{v}_\perp), \subseteq) \underset{\dot{\alpha}}{\overset{\dot{\gamma}}{\leftrightarrows}} (\mathbb{E}\mathbb{v}^\sharp, \dot{\preccurlyeq})$$

- Intuitively, the implication $\dot{\alpha}(P) \dot{\preccurlyeq} P^\sharp \Rightarrow P \subseteq \dot{\gamma}(P^\sharp)$ states that $\dot{\alpha}(P)$ is a sound over-approximation of the property P; and,

Interval abstraction: Galois connection

- The abstraction/concretization functions $(\dot{\alpha}, \dot{\gamma})$ are closely related in that, for every $P \in \wp(\mathbb{E}\mathbb{v}_\perp)$ and $P^\sharp \in \mathbb{E}\mathbb{v}^\sharp$,
$$\dot{\alpha}(P) \dot{\preccurlyeq} P^\sharp \Longleftrightarrow P \subseteq \dot{\gamma}(P^\sharp).$$

- Which is the definition of a Galois connection (GC), which we write:
$$(\wp(\mathbb{E}\mathbb{v}_\perp), \subseteq) \underset{\dot{\alpha}}{\overset{\dot{\gamma}}{\leftrightarrows}} (\mathbb{E}\mathbb{v}^\sharp, \dot{\preccurlyeq})$$

- Intuitively, the implication $\dot{\alpha}(P) \dot{\preccurlyeq} P^\sharp \Rightarrow P \subseteq \dot{\gamma}(P^\sharp)$ states that $\dot{\alpha}(P)$ is a sound over-approximation of the property P; and,
- the converse implication $\dot{\alpha}(P) \dot{\preccurlyeq} P^\sharp \Leftarrow P \subseteq \dot{\gamma}(P^\sharp)$ states that $\dot{\alpha}(P)$ is the *best* sound overapproximation of the property P.

Interval abstraction: Galois connection

- The abstraction/concretization functions $(\dot{\alpha}, \dot{\gamma})$ are closely related in that, for every $P \in \wp(\mathbb{E}\mathbb{v}_\perp)$ and $P^\sharp \in \mathbb{E}\mathbb{v}^\sharp$,
$$\dot{\alpha}(P) \dot{\preccurlyeq} P^\sharp \Longleftrightarrow P \subseteq \dot{\gamma}(P^\sharp).$$

- Which is the definition of a Galois connection (GC), which we write:
$$(\wp(\mathbb{E}\mathbb{v}_\perp), \subseteq) \underset{\dot{\alpha}}{\overset{\dot{\gamma}}{\leftrightarrows}} (\mathbb{E}\mathbb{v}^\sharp, \dot{\preccurlyeq})$$

- Intuitively, the implication $\dot{\alpha}(P) \dot{\preccurlyeq} P^\sharp \Rightarrow P \subseteq \dot{\gamma}(P^\sharp)$ states that $\dot{\alpha}(P)$ is a sound over-approximation of the property P; and,
- the converse implication $\dot{\alpha}(P) \dot{\preccurlyeq} P^\sharp \Leftarrow P \subseteq \dot{\gamma}(P^\sharp)$ states that $\dot{\alpha}(P)$ is the *best* sound overapproximation of the property P.
- So, GCs model the over-approximation of program semantics.

Interval abstraction: example

Going back to our example program P:

```
n := 1000;
while (0 < n) do
  n := n - 1
```

with $\sigma^\sharp = \{\mathtt{x} \mapsto [-\infty, +\infty]\}$, the abstract semantics would look like:

$$
\begin{aligned}
\mathcal{S}^\sharp[\![\mathtt{P}]\!]\sigma^\sharp &= (\mathcal{S}^\sharp[\![\mathtt{while\ (0<n)\ do\ n := n-1}]\!]\sigma^\sharp \circ \mathcal{S}^\sharp[\![\mathtt{n := 1000}]\!])\sigma^\sharp \\
&= \mathcal{S}^\sharp[\![\mathtt{while\ (0<n)\ do\ n := n-1}]\!]\{\mathtt{x} \mapsto [1000, 1000]\} \\
\ldots &\quad \{\mathtt{x} \mapsto ([1000, 1000] - [1, 1])\} \\
\ldots &\quad \{\mathtt{x} \mapsto ([999, 999] - [1, 1])\} \\
\ldots &\quad \textit{fixpoint convergence (using widening: pushing the interval ends} \\
&\quad \textit{to the constants in the program)} \\
&= \{\mathtt{x} \mapsto [0, 1000]\}
\end{aligned}
$$

Then, $\dot{\gamma}(\{\mathtt{x} \mapsto [0, 1000]\}) = \{\{\mathtt{x} \mapsto 0\}, \{\mathtt{x} \mapsto 1\}, \ldots\}$, which is weaker than $\mathcal{S}^*[\![\mathtt{P}]\!]\{\mathtt{x} \mapsto \mathbb{Z}\}$.

Interval abstraction: caveats

- The design of our abstract domain is rather imprecise.
- We can improve it by introducing context-sensitive semantics that takes into account the conditions when branching.

Conclusions

- We have seen a little and condensed appetizer of formal semantics and their approximation using abstract interpretation.
- For following sessions, I'd like to focus more on the implementation part, and fixpoint abstraction.
- Maybe Jurjo could also talk about domain combination (reduced product, Nelson-Oppen, . . .).
- Quoting Cousot's book:

  "Abstract interpretation provides a unified framework to understand formal methods, semantics design, specifications, proof methods, data-flow analysis, model checking, typing, static analysis, ... all of them described by a formal language."

Abstract Interpretation & its Application to Semantics Approximation

An Introduction

Marco Ciccalè

marco.ciccale@imdea.org

Universidad Politécnica de Madrid (UPM), Madrid Spain
IMDEA Software Institute, Madrid, Spain

Aut. Coffee – March 12, 2026

References I

Cousot, P. (2021).
Principles of Abstract Interpretation.
MIT Press.

Hoare, C. A. R. (1969).
An axiomatic basis for computer programming.
Communications of the ACM, 12(10).

Plotkin, G. D. (1981).
A structural approach to operational semantics.
Technical Report DAIMI FN-19, Computer Science Department, Aarhus University, Aarhus, Denmark.
Reprinted with corrections in Journal of Logic and Algebraic Programming, 60–61:17–139, 2004.

Scott, D. and Strachey, C. (1971).
Toward a mathematical semantics for computer languages.
Technical Report PRG06, OUCL.